

Ejercicio 01: Children Básico

Objetivo

Dominar el uso del prop `children` con tipado TypeScript para crear componentes contenedores flexibles.

Conceptos Clave

- Prop `children` como mecanismo de composición
- Tipado con `ReactNode` y `PropsWithChildren`
- Patrones de componentes contenedores
- Renderizado condicional de `children`

Duración Estimada

45 minutos

Instrucciones

Paso 1: Componente Card Básico

Aprenderás a crear un componente `Card` que acepta cualquier contenido como `children`.

Abre `starter/Card.tsx` y descomenta la sección correspondiente.

```
// QUÉ: Interface que define props con children tipado
// PARA: Permitir que Card acepte cualquier contenido React válido
// IMPACTO: TypeScript valida que children sea del tipo correcto
interface CardProps {
  children: ReactNode;
  className?: string;
}
```

Paso 2: Componente Alert con Variantes

Crearás un componente `Alert` que muestra `children` con diferentes estilos según la variante.

Abre `starter/Alert.tsx` y descomenta la sección correspondiente.

```
// QUÉ: Union type para variantes predefinidas
// PARA: Limitar las opciones de estilo a valores conocidos
// IMPACTO: IDE sugiere opciones válidas, errores en tiempo de compilación
type AlertVariant = 'info' | 'success' | 'warning' | 'error';
```

Paso 3: Componente Container Responsivo

Implementarás un `Container` que envuelve `children` con estilos de layout responsivo.

Abre `starter/Container.tsx` y descomenta la sección correspondiente.

```
// QUÉ: PropsWithChildren añade children?: ReactNode automáticamente
// PARA: Reducir código repetitivo al definir interfaces con children
// IMPACTO: Menos boilerplate, children siempre opcional
type ContainerProps = PropsWithChildren<{
  maxWidth?: 'sm' | 'md' | 'lg' | 'xl';
}
```

```
    centered?: boolean;
  }>;
```

Paso 4: Componente Conditional Wrapper

Crearás un componente que condicionalmente envuelve sus children.

Abre `starter/ConditionalWrapper.tsx` y descomenta la sección correspondiente.

```
// QUÉ: Props para renderizado condicional de wrapper
// PARA: Envolver children solo si se cumple una condición
// IMPACTO: Evita duplicación de código y JSX anidado innecesario
interface ConditionalWrapperProps {
  condition: boolean;
  wrapper: (children: ReactNode) => ReactElement;
  children: ReactNode;
}
```

Paso 5: App de Demostración

Combina todos los componentes creados en una aplicación de demostración.

Abre `starter/App.tsx` y descomenta las secciones para ver los componentes en acción.

✓ Criterios de Evaluación

Criterio	Puntos
Card implementado con tipado correcto	5
Alert con variantes funcionales	5
Container con PropsWithChildren	5
ConditionalWrapper funcionando	3
Código limpio y bien comentado	2
Total	20

📁 Estructura del Ejercicio

```
ejercicio-01-children-basico/
├── README.md
├── starter/
│   ├── Card.tsx
│   ├── Alert.tsx
│   ├── Container.tsx
│   ├── ConditionalWrapper.tsx
│   └── App.tsx
└── solution/
    ├── Card.tsx
    ├── Alert.tsx
    ├── Container.tsx
    ├── ConditionalWrapper.tsx
    └── App.tsx
```

Recursos

- [React TypeScript Cheatsheet - Children](#)
- [React Docs - Passing Children](#)